Lightweight reranking for language model generations

Siddhartha Jain*
AWS AI Labs
siddjin@amazon.com

Xiaofei Ma
AWS AI Labs
xiaofeim@amazon.com

Anoop Deoras
AWS AI Labs
adeoras@amazon.com

Bing Xiang
AWS AI Labs
bxiang@amazon.com

arXiv:2307.06857v3 [cs.AI] 11 Jan 2024

Abstract

Large Language Models (LLMs) can exhibit considerable variation in the quality of their sampled outputs. Reranking and selecting the best generation from the sampled set is a popular way of obtaining strong gains in generation quality. In this paper, we present a novel approach for reranking LLM generations. Unlike other techniques that might involve additional inferences or training a specialized reranker, our approach relies on easy to compute pairwise statistics between the generations that have minimal compute overhead. We show that our approach can be formalized as an extension of self-consistency and analyze its performance in that framework, theoretically as well as via simulations. We show strong improvements for selecting the best k generations for code generation tasks as well as robust improvements for the best generation for the tasks of autoformalization, summarization, and translation. While our approach only assumes black-box access to LLMs, we show that additional access to token probabilities can improve performance even further.

1 Introduction

The rapid advancement and remarkable achievements of generative large-scale pre-trained language models (LLMs) have brought about a revolutionary transformation in the field of natural language processing (NLP). These models have demonstrated significant enhancements in various NLP applications, such as machine translation, summarization, and code generation. Individual generations sampled from the models often yield high-quality results. However the quality of generated outputs can exhibit considerable variability. Multiple output samplings for the same input can produce certain generations which are of substantially higher quality than the quality of the average generation from the model.

Several approaches have been proposed to exploit this phenomenon. One strategy involves improving the underlying models themselves to make the quality of the average generation consistently better. This can be achieved by taking existing model generations, ranking them based on a human feedback, automated evaluation metrics like BLEU score, or execution feedback in case of code. The ranked generations can then be finetuned on directly or can be used to train a reward model that can be used in an RL loop (Hsieh et al., 2023; Ouyang et al., 2022; Ho et al., 2022; Polu et al., 2022; Liu and Liu, 2021; Ouyang et al., 2022). Another common approach is best-of-n sampling or *reranking*. In this approach, the underlying model is not touched – we instead take multiple samples from the model and select the best one post-facto using a reranking method (Ravaut et al., 2022; Jiang et al., 2022b; Zhang et al., 2022; Chen et al., 2021; Shi et al., 2022; Li et al., 2022; Mizumoto and Matsumoto, 2016; Uesato et al., 2022). While this approach can often given strong improvements, most extant reranking techniques involve computationally intensive or cumbersome methods to compute the ranking criterion. These include methods like training an auxiliary model as a reranker, evaluating the probability of the query given the generated answer (query likelihood) but at the price of doubling the inference cost, etc. In case of code generation models, another alternative is executing the generated code on unit tests. While such an approach has been applied in various models such as AlphaCode (Li et al., 2022) which is targeted towards contest coding problems, it becomes much less feasible as you move past the contest coding setting due to the complexity of setting up the build environment for arbitrary code as well as sandboxing it appropriately.

Recently, a simple approach, called self-consistency was proposed for selecting the best answer from multiple generations (Wang et al., 2022) for tasks where the set of possible answers is small – for example multiple choice questions or math word problems where there is a *unique* answer consisting of a single or a very limited number of tokens. In that paper, the authors sample multiple chain-of-thought generations from the LLM, extract the predicted answer at end each generation and select the answer with the most number of votes. The motivation behind this is the observation that you

*Corresponding author

can take *different* reasoning paths to get to the same answer. Thus the method aims to *marginalize* over multiple different reasoning paths and rank the answers based on their *marginal* probability rather than their probability conditioned on a single reasoning path. While they achieve substantial improvements over existing baselines, it is not immediately clear how to apply this to open-ended generation tasks like code generation, summarization, or translation - where there is often no chain-of-thought or reasoning path to marginalize over, nor is there necessarily a unique correct answer.

We start off with two key observations – (1) We can have semantically equivalent or near-equivalent generations that are nevertheless not exact matches. These are one subset of generations we can marginalize over (2) For open-ended tasks, a generation can encompass multiple elements. For summarization, there might be multiple relevant facts in the text that a good summary should mention. For code, there might be multiple branch conditions that need to be present to generate a correct implementation. Our generation set could be structured such that while different generations include a different subset of elements (different facts in case of summarization or different branch conditions in case of code), we have only a single generation that contains *all* of the relevant elements. In this case, simply marginalizing over semantically equivalent generations would not be sufficient as there is no semantically equivalent generation for the optimal generation.

We develop these two observations in the next section into a minimal overhead reranking method for such open-ended tasks which does not require access to token probabilities.

Concretely, our contributions are as follows –

- We connect the above two observations with the notion of self-consistency. Based on that connection, we then proceed to design an effective minimal overhead reranker which does not require access to token probabilities. We show that the reranking methods utilized in previous works Shi et al. (2022); Li et al. (2022) can also be understood within the same conceptual framework.

- We conduct simulations where we demonstrate that our framework is capable of recovering the best or near-best generation in many cases. We also prove some properties of our methodology that provide guarantees on its effectiveness.

- We extend our reranker to *optionally* account for token log probabilities (if they are provided) and show that doing so gives a much better reranker than just mean log probability reranking (which also requires access to token log probabilities)

- Empirically, while our focus is on code generation tasks where we demonstrate significant gains, we also experiment with the tasks of autoformalization, summarization, and translation and find that our approach leads to non-trivial though smaller gains there.

- As our method is based on pairwise similarity between generations, we are able to leverage that property to improve ranked best-of-k performance for different values of k.

- We conduct multiple experiments ablations to understand the effect of various experimental settings.

The rest of the paper is organized as follows. In Section 2 we present our motivation. In Section 3 we present our method and the similarity function. In Section 4, we present and discuss our experimental results. In Section 5, we describe the related work and we finally conclude in Section 6.

2 Motivation

Consider the following coding problem from the MBPP dataset –

```
def remove_dirty_chars(string, second_string):
    """
    Write a function to remove characters
        from the first string which are
        present in the second string.
    >>> remove_dirty_chars("probasscurve",
        "pros")
    'bacuve'
    >>> remove_dirty_chars("digitalindia",
        "talent")
    'digiidi'
    >>> remove_dirty_chars("exoticmiles",
        "toxic")
    'emles'
    """
```

A solution to the above problem would be semantically equivalent to "iterate over the string skipping characters in second_string and then convert the result back to a string and return". Two parts of the semantic meaning of this solution could then be (1) the return type should be a string (2) when iterating through the string, any character in second string has to be skipped over. These observations can be converted into *predicates* for the generations. Specifically, for this prompt, we can define the predicates (1) $p_1 =$ is the return value of the generated program a string? (2) $p_2 =$ in the generated program, are all characters in `second_string` skipped over in the return string? These predicates capture properties of the semantic meaning of the generations. We sample three generations for the above prompt resulting in the following generations:

```
# First generation (Incorrect)
    return [char for char in string if char
        not in second_string]

# Second generation (Incorrect)
    return ''.join([char for char in string])

# Third generation (Correct)
    return ''.join([char for char in string
        if char not in second_string])
```

Now if we were able to evaluate the above predicates at inference time on the generations, we would be able to detect that generation 3 is the only one that satisfies both and is thus an optimal generation. However generating the relevant predicates, and then generating code to evaluate arbitrary predicates on code that confirms to the given natural language specification with high precision is an unsolved problem.

Is there a way transform the problem into something more tractable? Let us look at the votes each predicate gets from the different generations (i.e. on how many generations the predicate evaluates to true). p_1 gets $2/3$ votes (from the 2nd and 3rd generation) and thus the *majority vote* is that it should be true. p_2 gets $2/3$ votes (from the 1st and 3rd generation) and thus the majority vote again says it should be true. Generation 3 is the only one that agrees with the majority vote for p_1, p_2 and is thus the consensus choice.

In fact, we do not even have to do the step of first counting votes for p_1, p_2 to figure out what their majority vote value is! We can just compute how much a generation agrees with the other 2 generations on the evaluation for p_1, p_2. To elaborate, generation 1 agrees with generation 3 on p_2 but not p_1. It does not agree with generation 2 on anything. Thus it has a total agreement score of 1. Similarly generation 2 also has an agreement score of 1. Generation 3 however agrees with generation 1 on p_2 and with generation 2 on p_1 resulting in an agreement score of 2. Thus generation 3 has the highest agreement with all other generations and is the consensus choice. This transformation is depicted in Figure 1.

There are a couple of points to note regarding the above. (A) The answer we want for both predicates ends up being also what the majority vote predicts. This is the connection to self-consistency that we will formalize shortly. (B) While the above procedure handles Observation (2) in the mentioned in the introduction, does it also handle Observation (1)? Note that if we had a 4th generation

```
# Fourth generation (Correct)
    new_str = ''
    for c in string:
        if c not in second_string
            new_str += c
    return new_str
```

this would also agree with the 3rd generation on p_1, p_2. Thus this generation is semantically equivalent to the 3rd generation with respect to p_1, p_2. Including this generation would lead to a tie between the 3rd and 4th generation with respect to the scheme above. Thus the above procedure can also account for generations that are semantically equivalent.

As mentioned before, at inference time however, we have access neither to such predicates nor to what their evaluation on the different generations might be. However, as the previous example showed, we do not actually even need an evaluation of the predicates on the generations. We only actually need an understanding of how much a pair of generations agree on relevant predicates. In the next section we will describe simple similarity functions to approximate this agreement without any knowledge of the possible predicates and later show their empirical effectiveness. For now, we attempt to formalize the above intuition, connect it with self-consistency, and give it theoretical grounding.

Let $\mathbf{v}$ be a vector of length k where each element represents a predicate and the value represents the property's value that you want the generation to have. For example, for the example above, $\mathbf{v}$ would be of length 2 with a value of 1 in both. In general, we do not restrict v to be just a binary vector as some predicates can have multiple possibilities as an answer. Let $\mathbf{u}_i$ be the vector for the ith generation representing the predicates' values for that generation. We now make the *self-consistency* assumption which is that for each individual predicate, the most frequent response is assumed to be correct. Formally if $\mathbf{v}^l$ can take on m_l values $1, \ldots, m_l$ and without loss of generality, $\mathbf{v}^l = 1$, then $l = \arg\max_j \sum_{i=1}^{n} \mathbb{I}(u_i^l = j)$.

Now as mentioned, at inference time, we may not have access to either the predicates or their values for different generations. Thus we only assume access to the *agreement* between the different generations on the predicates' values (later on we will approximate this with similarity functions between generations). In particular we assume we know what the pairwise fractional agreement on the predicates is between generations denoted as $a(\mathbf{u}_i, \mathbf{u_j}) = \frac{1}{k} \sum_{t=1}^{k} \mathbb{I}(\mathbf{u}_i^t = \mathbf{u}_j^t) \forall i, j \in [1, n]$ where i indexes the generations and t the predicates. We then try to identify a generation i such that the average pairwise fractional agreement for that generation with all other generations is maximized – i.e. $a(\mathbf{u}_i, \mathbf{v})$ is maximized.

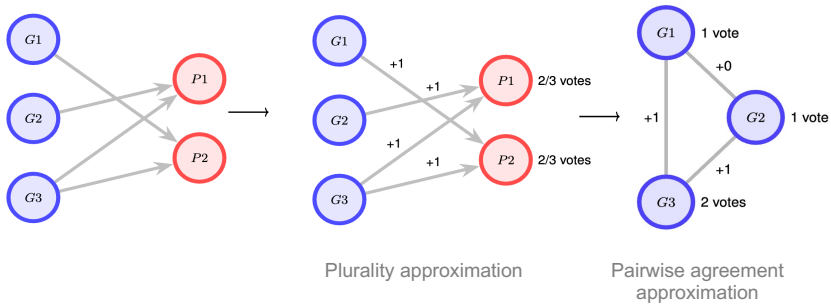


Figure 1: On the left we have the original setup where we have predicates which we know the optimal generation should satisfy and which we can evaluate on the generations. In the middle, we drop the assumption that we know whether the optimal generation should satisfy the predicates or not. On the right, we drop the assumption that we need to evaluate the predicates on the different generations – only assuming we know on how many predicates a pair of generations agree

Given this problem formulation and selection criterion, we can establish the following:

Theorem 2.1. *For $k = 1$, we always recover the best $\mathbf{u}$. However for $k > 1$, it is not guaranteed.*

Informally if there is only a single predicate we care about, then we can always recover the optimal generation. Moreover:

Theorem 2.2. *If there exists $\mathbf{u}_b = v$, then $b = \arg\max_i \frac{1}{n-1} \sum_{i \neq j} a(\mathbf{u}_i, \mathbf{u_j})$.*

Informally this says that if a generation g exists such that its predicate vector perfectly aligns with the optimal vector v, selecting the generation with the highest average fractional agreement with other generations will pick g.

The previous theorem only works if the optimal generation is part of the set of generations. What if that is not the case (as is likely)? The next theorem gives upper and lower bounds on the fractional agreement we can expect. Now if we assume that $\mathbf{u}_i^j$ are iid from $Bernoulli(p_j)$ and n is the number of generations, then we can show that

Theorem 2.3. $\mathbb{E}[\sum_j^k \mathbf{u}_b^j] \leq \sum_{j=1}^k p_i + \sqrt{\frac{k \log n}{2}}$

where $\mathbf{u}_b$ denotes the sequence selected by our method.

All proofs for these theorems are presented in the Supplement. While the theorems give some guarantees on the performance, the bounds in Theorem 2.3 are still not very tight. Furthermore, They are only for the case where the predicates are binary valued. To further substantiate our selection criterion — picking the generation with the highest average fractional agreement with all other generations — we conducted a simulation. The setup is as follows – we fix the number of predicates (length k of the vector $\mathbf{v}$ in the above notation) as well as the number of values the predicate can take. We then simulate the generations predicate evalutions by assuming a generation has an equal chance of having an value the predicate can take. However we force the self-consistency constraint that for every predicate, the *plurality* of generations should have the property that matches the predicate value in v. The results are in the Supplement. Our findings show that our method successfully recovers the best generation the majority of the time, significantly outperforming random selection. Moreover, on average, the generation we recover demonstrates nearly 100% agreement with best generation, even in cases where we do not select the best generation. The full details are in the Supplement.

3 Method

As previously mentioned, we may not have the capability to compute predicates at inference time, thereby rendering the computation of the exact fractional agreement with $\mathbf{v}$ i.e. $a(\mathbf{u}, \mathbf{v})$, unattainable. However as we found out in the last section, choosing the generation that has the maximum average fractional similarity with all other generations can be a good approximation. However as we may not have predicates at inference time, we cannot always compute that either. Intuitively however, if two generations are more similar to each other – for an appropriate definition of similarity – then they will tend to agree more on *any* possible predicates. Surprisingly, we find that a very simple similarity function, which we will define shortly, is sufficient for our purposes.

Once we have our similarity function, we can define a generalized self-consistency score $GSC_{Sim}(i)$ for each generation i, given by $\frac{1}{M-1} \sum_{j=1, j \neq i}^{M} Sim(i, j)$. Here, Sim denotes the similarity function, and M represents the number of generations.

For generations with unique answers, if we have:

$$Sim(i, j) = \mathbb{I}(\text{Answer in generation } i \text{ is an exact match with Answer in generation } j)$$

this is equivalent to the self-consistency criterion. Two other reranking methods - MBR-Exec (Shi et al., 2022) and AlphaCode (Li et al., 2022) - can be viewed in terms of the same formulation with the difference being that of the similarity function. MBR-Exec *executes* model generated code. It then

defines gives a similarity score of 1 if a pair of programs agree on *all* unit tests and 0 otherwiseFor each program, they sum the similarity vs all other programs and pick the program with the highest similarity. Similarly AlphaCode clusters its generated programs by executing them on test cases and selecting a program from the largest cluster – with two programs cluster together if they agree on on all test cases. This is conceptually equivalent to what MBR-Exec does. We give further evidence that this is a useful way to frame self-consistency by evaluating another OpenAI Ada embedding based similarity function (Section H in the Supplement). While its performance is promising, as the similarity function is a lot more heavyweight requiring a separate embedding model, we chose not to explore it further.

One straightforward way to encode a generation is by using a binary vector that denotes the presence or absence of an n-gram. Surprisingly, we find this simple encoding to be sufficient for defining a robust similarity function. For open-ended generation, we define our similarity function as follows. For each generation we define a vector $\mathbf{v}$ of size $|V|$ where V is set of all possible n-grams for $n = 1$ to $n = K$ where K is a hyperparameter. For the experiments in this paper, we simply use $K = 1$. We show in Section F, increasing K can be helpful though only up to a point. Each element i of $\mathbf{v}$ is simply whether token i is present in the generation or not. We then take the inner product between two such vectors as similarity. We call this the Ngram consistency score (NCS) and refer to the $K = 1$ version as the Unigram consistency score (UCS). Figure 2 shows a visualization of $\mathbf{v}$ for an example sentence. Formally

$$UCS(i, j) = \frac{1}{|V|}\mathbf{v}_i \cdot \mathbf{v}_j$$

where

$$\mathbf{v}_i^j = \mathbb{I}(t_j \in g_i)$$

where t_j is the jth token and g_i the ith generation. This definition only requires model generations and incurs minimal computational overhead – we only need to compute the unigram overlap instead of training an auxiliary model, running generated programs, or performing additional inferences using the same model (which will increase compute cost as well as latency). Notably, we don't normalize the inner product by the norm of the vectors. This is a deliberate design choice that encourages more diverse sequences, in response to known issues of neural generation models producing degenerate and repetitive sequences (Zhang et al., 2022; Welleck et al., 2019). We delve into this topic in Section I in the Supplement.

When token probabilities are available, we can leverage them to improve our approach. Intuitively, if a generation has a low token probability for the generated token, then finding a match for that that token should count for less. In accordance with this intuition, we introduce two further variants. First we modify the definition of $\mathbf{v}$ as follows

$$\mathbf{v}_i^j = \begin{cases} \frac{1}{c_j^i}\sum_k^{c_j^i} p(t_j^{i,k}) & \text{if } t_j \in g_i, \\ 0 & \text{otherwise} \end{cases}$$

where c_i^j is the number of times token t_j appears in generation i and $p(t_j^{i,k})$ is the token probability of the jth token's kth appearance in generation i. We call this the weighted n-gram consistency score (WUCS). Figure 2 has a visualization for an example sentence.

The mean log probability of a sequence is an oft-used ranking method. We can combine it with WUCS by further weighting each generation by the per token probability as follows – for a generation i, Consensus-WUCS $= WUCS \cdot e^{(1/|g_i|)\cdot p(g_i)}$ where g_i is the length of generation i.

Finally, to rank the generations, we employ $\arg\max_i GSC_{Sim}(i)$ where Sim can take the form of UCS, WUCS, or Consensus-UCS.

3.1 Extending to ranked *pass@k*

A common evaluation metric for code generation problems is ranked *pass@k* wherein we assess whether *any* program among the top k selected programs (selected from a larger set) can pass all the given unit tests for that problem. Typically, the top k generations are selected based on a predetermined ranking. However, with our similarity-based metric, we can apply a more nuanced approach.

For a particular problem, if the highest-ranked generation for a specific prompt is correct, we have already succeeded. We would only need to utilize the remaining generations in our k-budget if the top-ranked generation does not pass some unit test case. In this event, we could consider the top-ranked generation as a hard negative and select the next generation that exhibits lower similarity to the top-ranked generation.

More specifically, if we have selected programs $S_{k'}$ so far ($|S_{k'}| = k' < k$, then we modify the GCS function to select the $k' + 1$th item in the list. In particular, we compute

$$GCS_{sim}^{ranked} = \frac{1}{n-1}\Big(\sum_{j \notin S_{k'}} sim(i, j) - \sum_{j \in S_{k'}} sim(i, j)\Big)$$

Note that for $k = 1$, GCS and GCS^{ranked} are equivalent. We demonstrate in Section 4.4 that GCS_{Sim}^{ranked} performs significantly better in ranking for *pass@k* where $k > 1$ than raw GCS. This approach leads to a more efficient utilization of the ranked generations, improving the overall effectiveness of the code generation task.

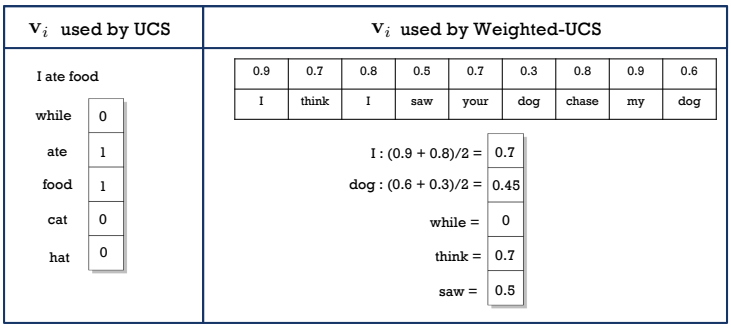


Figure 2: On the left, we depict the $\mathbf{v}_i$ corresponding to the sentence `I ate food` used by UCS. On the right we show the $\mathbf{v}_i$ corresponding to the sentence `I think I saw your dog chase my dog` used by Weighted-UCS

4 Results

We conducted experiments utilizing the Codex family of models, specifically Codex-davinci-001, Codex-davinci-002, and Codex-Cushman as well as Llama family of models. In addition we also evaluated GPT-J for Xsum, MiniF2F, and WMT14. Unfortunately due to the unexpected shutdown of the OpenAI API, we were unable to obtain results for Codex-001 and Codex-Cushman on the Xsum, MiniF2F, and WMT14 datasets. We evaluated these models on a range of datasets for code generation tasks – in particular on the HumanEval (Chen et al., 2021), MBPP, MBPP-sanitized (Austin et al., 2021) datasets for code generation. For the autoformalization of MiniF2F to Isabelle, we used the dataset provided by (Jiang et al., 2022a). For text summarization, we utilized the Xsum dataset (Narayan et al., 2018). For machine translation, we used the WMT14 French to English and German to English datasets (Bojar et al., 2014).

Our primary evaluation metric for code generation is ranked *pass*@1 where we rerank a sample set of generations and assess whether the top-ranked generation successfully passes all unit tests. We also evaluate with ranked *pass*@k for $k > 1$. For the MiniF2F autoformalization task, we measure the quality using the BLEU score, following Wu et al. (2022). For Xsum we use the Rouge-2 and Rouge-L scores for evaluation. For all code generation datasets, we sample 125 generations from the models which serves as our dataset for the different experiments For MiniF2F and Xsum, we sample 50 generations from the model. Unless otherwise specified, for all experiments, we use the Codex-davinci-002 model. Following (Shi et al., 2022; Zhang et al., 2022), we perform bootstrap sampling 50 times with a sample size of 25 to generate the results.

Our baselines are Random selection, Ranking by mean log probability, Ranking using Medoid in our confidence weighted unigram space, and for code generation - ranking using the Coder Reviewer Ranker method (Zhang et al., 2022). **A full description of the datasets, experiments, and the baselines is in the Supplement. We also have several additional results in the Supplement.**

4.1 GSC scores are higher for correct answers

As a sanity check, we first evaluate whether the GSC scores are indeed higher for the correct generations [1] The results are in Table 8 in the Supplement. The ratios are consistently > 1 for all models except for the UL2-20B model for which they still remain very close to 1.

4.2 UCS shows strong improvements for Code Generation

As shown in Tables 1 and 2 (Supplement), the application of the UCS, WUCS, and Consensus-WUCS methods leads to substantial improvements in the accuracy as well as mean reciprocal rank of code generation across various models and datasets.

In the HumanEval dataset, UCS variants consistently outperform the traditional methods, namely Random and mean log probability. For instance, the Codex002 model exhibits a substantial accuracy improvement from 0.435 (Random) to 0.568 (Consensus-WUCS). Even the less performing models, such as Llama-13B and Llama-30B, exhibit noticeable accuracy gains when our proposed methods are employed.

Similar trends are observed in the MBPP-S and MBPP datasets. UCS, WUCS, and Consensus-WUCS consistently improve the accuracy across all models. Specifically, the Consensus-WUCS method consistently dominates Random and mean log probability ranking in all categories, and almost always outperforms WUCS as well. Of particular note is the performance of WUCS, which surpasses the mean log probability method in every model and dataset combination. In fact it is the best method for all dataset and model combinations except LLama-13B model for MBBP and MBPP-S. UCS, which does not require token probabilities and

[1] We used the generations in (Li et al., 2022) provided by them as part of their Supplementary Material.

relies only on the generations, also demonstrates a consistent superiority over the random reranking.

Consensus-WUCS and WUCS are also almost always better than the Medoid based approach with Consensus-WUCS outperforming it 13/15 times. A discussion of the mean reciprocal ranking performance is deferred to the Supplement but the trend is similar.

4.3 UCS shows consistent improvements for non-coding tasks

Next, we evaluated the performance of UCS, WUCS, and Consensus-WUCS on the non-coding tasks. In the case of the MiniF2F dataset, evaluated using the BLEU metric, Consensus-WUCS outperforms all other methods for the Codex002 model except for Medoid. For the Llama-13B, Llama-30B, and GPT-J models, the top performers are closely matched, with Consensus-WUCS, WUCS, and UCS all delivering competitive scores.

Turning to the Xsum dataset, we see a similar trend. For the Rouge-2 metric, Consensus-WUCS achieves the highest score for the Codex002 and both LLama models, and ties for the best score with WUCS for the Llama-13B model. In the GPT-J model, UCS performs slightly better than the WUCS and Consensus-WUCS. Nonetheless, all these methods surpass Random, and Mean-logp reranking methods and almost always surpass Medoid.

With the Rouge-L metric, UCS variants show the best performance for the all models except Codex002. For the Llama-30B model, WUCS and Consensus-WUCS share the top spot, while UCS achieves the best score for the GPT-J model. Once again, these methods generally outperform Medoid, Random, and Mean-logp reranking methods.

For the WMT14 translation dataset, Conensus-WUCS is the best for all models and both tasks except for the German to English Llama-13B model. WUCS also shows strong performance, matching Consensus-WUCS score on 3 model-task combinations. UCS is also consistently better than random selection.

In total, Consensus-WUCS gets the top spot in 12/20 comparisons, WUCS in 7/20, UCS in 3/20, and Medoid in 5/20 primarily due to MiniF2F.

4.3.1 Cause of smaller performance improvements for non-coding tasks

We took the top 3 and bottom 3 generations for coding and non-coding tasks and computed the unigram overlap within each set. The results are in Table 6 (Supplement). The ratio of the overlap for coding tasks was a lot higher than that for non-coding tasks giving a hint as to why we see a much stronger improvement for coding tasks. This means that if a unigram is not shared between two generations, that gives a lot more information about whether two generations are semantically far apart for coding tasks versus non-coding tasks. Thus computing the unigram overlap statistic is a lot more informative for code generation tasks vs non-code generation tasks which could be the reason behind the smaller gains for non-coding tasks. However we want to note that while the gains are smaller, they are similar to gains that that past published papers report for such metrics and importantly, the gains are robust across different tasks and models.

4.4 GCS^{ranked} comparison

In Figure 3 (Supplement), we show how the model performance changes as k for $pass@k$ increases. We compare GCS vs GCS^{ranked}. While the performance of GCS declines quickly, GCS^{ranked} maintains good performance even at larger values of k for all code generation datasets.

5 Related Work

5.1 Auxiliary reranker

In Mizumoto and Matsumoto (2016), they use a perceptron based reranker to rerank model generated translations. SummaReranker (Ravaut et al., 2022) use mixture of experts training to train their reranker to optimize for multiple automated evaluation metrics (like ROUGE or BLEU score) at once. PairReranker (Jiang et al., 2022b) uses automated evaluation metrics to rank model generations and then select the top few best and worse and train a model to classify the better summary between a pair of summaries. All of the previous reranking methods however require training an auxiliary model.

5.2 Code generation reranking

There have also been multiple reranking proposals for code generation in particular. A unique characteristic of code (as oppposed to text) is that code can be executed. Thus several methods have tried to exploit that property for reranking. MBR-Exec (Shi et al., 2022) and AlphaCode (Li et al., 2022) both execute the generated codes on unit tests. They rank the different codes according to how many other codes are semantically equivalent to them (i.e. have the same results on the given unit tests). CodeT (Chen et al., 2022) uses LLMs to generate both code and candidate unit tests. They then find sets of generated codes such that the product of the size of the set and the size of the *unit test* set the codes agree on is maximized. More recently, Coder-Reviewer Ranker (Zhang et al., 2022) applies the well known Maximum Mutual Information objective (Li et al., 2015) to code generating LLMs by using the strong few shot and zero prompting capabilities of LLMs to obtain the query likelihood.

No logprobs used			logprobs used			
	Random	**UCS**	**Medoid**	**Mean-logp**	**WUCS**	**Consensus-WUCS**
	HumanEval					
Codex002	0.435	*0.539*	0.437	0.539	**0.558**	***0.568***
Codex001	0.345	*0.402*	0.354	0.408	**0.426**	***0.445***
Code-Cushman	0.311	*0.353*	0.335	0.355	**0.373**	***0.381***
Llama-13B	0.142	*0.177*	0.17	0.17	**0.187**	***0.192***
Llama-30B	0.207	*0.257*	0.225	0.228	**0.263**	***0.267***
	MBPP-S					
Codex002	0.55	*0.572*	**0.583**	0.57	0.580	***0.589***
Codex001	0.494	*0.523*	0.532	0.515	**0.535**	***0.546***
Code-Cushman	0.436	*0.457*	0.467	0.456	**0.472**	***0.488***
Llama-13B	0.247	*0.261*	**0.284**	0.27	0.266	***0.277***
Llama-30B	*0.325*	0.253	0.357	0.348	**0.363**	***0.373***
	MBPP					
Codex002	0.536	*0.58*	0.563	0.512	**0.587**	***0.594***
Codex001	0.475	*0.505*	0.505	0.503	**0.520**	***0.525***
Code-Cushman	0.305	*0.386*	0.343	0.319	**0.405**	***0.420***
Llama-13B	*0.185*	0.183	***0.202***	0.197	0.195	**0.199**
Llama-30B	0.262	*0.276*	0.276	0.273	**0.287**	***0.294***

No logprobs used			logprobs used			
	Random	**UCS**	**Medoid**	**Mean-logp**	**WUCS**	**Consensus-WUCS**
	MiniF2F					
Codex002	55.8	55.6	**58.2**	52.9	55.8	**56.2**
Llama-13B	24.3	24.6	**24.9**	24.2	24.7	**24.8**
Llama-30B	**26**	25.6	**26.4**	25.6	25.7	25.7
GPT-J	24.2	24.7	**24.8**	24	**24.8**	**24.8**
	Xsum Rouge2					
Codex002	19.7	21	**21.8**	21.4	21.5	**21.9**
Llama-13B	9.2	10.4	10.3	10.3	**10.6**	**10.6**
Llama-30B	10.7	12.1	12	12.2	**12.2**	**12.3**
GPT-J	6.5	**7.1**	6.9	6.6	**7**	6.9
	Xsum RougeL					
Codex002	33.9	34.8	**36.3**	35.1	35.3	**35.6**
Llama-13B	19.6	20.9	20.7	20.3	**21**	**20.9**
Llama-30B	21.4	23	22.7	22.8	**23.1**	**23.1**
GPT-J	17.2	**18**	17.5	16.6	**17.8**	17.5
	WMT14 French → English BLEU					
Codex002	34.7	36.2	35.9	**36.6**	36.5	**37**
Llama-13B	4.3	4.4	4.2	**4.5**	**4.5**	**4.6**
Llama-30B	3.9	**4**	**4**	**4**	**4.1**	**4.1**
GPT-J	3.8	**3.9**	3.8	**3.9**	**4**	**4**
	WMT14 German → English BLEU					
Codex002	30.7	31.6	31.2	**33.2**	32.1	**34**
Llama-13B	3.4	3.5	3.1	**4**	3.5	**3.6**
Llama-30B	3.7	3.7	3.5	**3.9**	**3.8**	**3.9**
GPT-J	3.1	**3.3**	**3.2**	**3.2**	**3.3**	**3.3**

Table 1: Accuracy of generated code for HumanEval, MBPP, MBBP-S as well as performance on Xsum, MiniF2F, WMT14 French to English, and WMT14 German to English datasets. All scores are out of 100. Best results are colored in **first**, **second**. Italics for best in category (logprobs used vs not).

6 Conclusion

We analyze the self-consistency method for problems that have fixed answers and develop a framework to extend it to open-ended generations. We establish connections between our framework and other code generation reranking functions and prove that if the optimal generation is present in our generation set, we can always recover it as well as prove bounds on how close we can get to the optimal generation under certain settings.

Our simulated tests reveal our ability to consistently recover the best or close to best possible generation in the set. We introduce several lightweight similarity functions and show that they give strong and consistent improvements over state of the art baselines. Notably, our Unigram Consistency Score (UCS) function, the most minimal of our similarity functions, requires only access to raw generations to effectively rerank. We show that the UCS variants uniformly enhance the performance of code and text generation and are competitive with strong baselines like Coder Reviewer Reranker despite them needing a lot more compute resources as well as time. For code geneartion, we also leverage the fact that our reranking metric is based on pairwise similarity to improve performance for pass@k for $k > 1$. Additionally, we conduct multiple variations on our primary experiments to ascertain the robustness and reliability of our performance.

7 Broader Impact and Limitations

As a paper that tries to improve the performance of Large Language Models (LLMs), it inherits the risk and rewards of LLMs in general. LLMs have shown themselves highly relevant and useful for a number of tasks but in particular code generation. Our

method shows particularly strong improvements for that task and thus we hope will have a broad impact. Nevertheless, we did not evaluate our method on whether it increases its propensity to select biased or toxic generations which we leave to future work.

References

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.

Ondřej Bojar, Christian Buck, Christian Federmann, Barry Haddow, Philipp Koehn, Johannes Leveling, Christof Monz, Pavel Pecina, Matt Post, Herve Saint-Amand, et al. 2014. Findings of the 2014 workshop on statistical machine translation. In *Proceedings of the ninth workshop on statistical machine translation*, pages 12–58.

Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. 2022. Codet: Code generation with generated tests. *arXiv preprint arXiv:2207.10397*.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Namgyu Ho, Laura Schmid, and Se-Young Yun. 2022. Large language models are reasoning teachers. *arXiv preprint arXiv:2212.10071*.

Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. 2019. The curious case of neural text degeneration. *arXiv preprint arXiv:1904.09751*.

Cheng-Yu Hsieh, Chun-Liang Li, Chih-Kuan Yeh, Hootan Nakhost, Yasuhisa Fujii, Alexander Ratner, Ranjay Krishna, Chen-Yu Lee, and Tomas Pfister. 2023. Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes. *arXiv preprint arXiv:2305.02301*.

Albert Q Jiang, Sean Welleck, Jin Peng Zhou, Wenda Li, Jiacheng Liu, Mateja Jamnik, Timothée Lacroix, Yuhuai Wu, and Guillaume Lample. 2022a. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs. *arXiv preprint arXiv:2210.12283*.

Dongfu Jiang, Bill Yuchen Lin, and Xiang Ren. 2022b. Pairreranker: Pairwise reranking for natural language generation. *arXiv preprint arXiv:2212.10555*.

Jiwei Li, Michel Galley, Chris Brockett, Jianfeng Gao, and Bill Dolan. 2015. A diversity-promoting objective function for neural conversation models. *arXiv preprint arXiv:1510.03055*.

Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097.

Yixin Liu and Pengfei Liu. 2021. Simcls: A simple framework for contrastive learning of abstractive summarization. *arXiv preprint arXiv:2106.01890*.

Tomoya Mizumoto and Yuji Matsumoto. 2016. Discriminative reranking for grammatical error correction with statistical machine translation. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1133–1138.

Shashi Narayan, Shay B Cohen, and Mirella Lapata. 2018. Don't give me the details, just the summary! topic-aware convolutional neural networks for extreme summarization. *arXiv preprint arXiv:1808.08745*.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744.

Stanislas Polu, Jesse Michael Han, Kunhao Zheng, Mantas Baksys, Igor Babuschkin, and Ilya Sutskever. 2022. Formal mathematics statement curriculum learning. *arXiv preprint arXiv:2202.01344*.

Mathieu Ravaut, Shafiq Joty, and Nancy F Chen. 2022. Summareranker: A multi-task mixture-of-experts re-ranking framework for abstractive summarization. *arXiv preprint arXiv:2203.06569*.

Freda Shi, Daniel Fried, Marjan Ghazvininejad, Luke Zettlemoyer, and Sida I Wang. 2022. Natural language to code translation with execution. *arXiv preprint arXiv:2204.11454*.

Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. 2022. Solving math word problems with process- and outcome-based feedback. *arXiv preprint arXiv:2211.14275*.

Martin J Wainwright. 2019. *High-dimensional statistics: A non-asymptotic viewpoint*, volume 48. Cambridge university press.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*.

Sean Welleck, Ilia Kulikov, Stephen Roller, Emily Dinan, Kyunghyun Cho, and Jason Weston. 2019. Neural text generation with unlikelihood training. *arXiv preprint arXiv:1908.04319*.

Yuhuai Wu, Albert Qiaochu Jiang, Wenda Li, Markus Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. 2022. Autoformalization with large language models. *Advances in Neural Information Processing Systems*, 35:32353–32368.

Tianyi Zhang, Tao Yu, Tatsunori B Hashimoto, Mike Lewis, Wen-tau Yih, Daniel Fried, and Sida I Wang. 2022. Coder reviewer reranking for code generation. *arXiv preprint arXiv:2211.16490*.

Supplementary Material

A Proofs

A.1 Proof of Theorem 2.1

Proof. This is true by definition for $k = 1$. For $k > 1$, let us assume that the number of categories $L = 3$. If the best generation g agrees with $\mathbf{v}$ on only one of the elements, then wlog, let that be the 1st one. Then the agreement score is $(p_1 + p_2')/2$ where $p_2' < p_2$. Let the agreement score for a generation g' that does not agree at all with $\mathbf{v}$ be $(p_1' + p_2'')/2$. However if for example $p_1 = 0.34, p_1' = 0.32, p_2' = 0.01, p_2'' = 0.32$, then g' will be selected over g. □

A.2 Proof of Theorem 2.2

Proof. It is true by assumption for $k = 1$. Assume it is true for $k = t$. Then that means that given the self consistency assumption that $a_t(\mathbf{u}_b, \mathbf{v})$ is the highest possible where a_t is the agreement until $k = t$. Then for $t + 1$, we know that $\sum_{i \neq b} \mathbb{I}(\mathbf{u}_b^{t+1} = \mathbf{u}_i^{t+1}$ is the highest (again by self-consistency assumption). Thus a_{t+1} is also the highest proving the theorem. □

A.3 Proof of Theorem 2.3

Formally, let $\mathbf{u}_i^j \sim Bernoulli(p_j)$. Let $b = \arg\max_i \sum^j p_j \cdot \mathbf{u}_i^j + (1 - p_j) \cdot (1 - \mathbf{u}_i^j) = \arg\max_i \sum^j \mathbf{u}_i^j \cdot (2p_j - 1)$ (i.e. the sequence selected by our method). Then we want a bound on $\mathbb{E}[\sum_j^k \mathbf{u}_b^j]$.

Proof. Let $q_i = \sum_j \mathbf{u}_i^j$. As all are iid, $\mathbb{E}[q_i] = \sum_j p_j$. We can upper bound this by upper bounding $\mathbb{E}[\max_i q_i]$. Note that $\mathbf{u}_i^j$ is subgaussian with parameter $1/2$ as it's bounded in $[0, 1]$. Thus q_i is subgaussian with parameter $\sqrt{k}/2$. Thus $\mathbb{E}[\max q_i - \mathbb{E}[q_j]] \leq \sqrt{\frac{k \log n}{2}} \implies \mathbb{E}[\max q_i] \leq \sum_i p_i + \sqrt{\frac{k \log n}{2}}$ where n is the number of generations (Wainwright, 2019)

□

B Simulation results

We setup our simulation as follows. Let d be the number of predicates, n the number of generations, and l the number of categories. Then for each predicate, we uniformly at random sample a categorical distribution and then generate $\mathbf{u}_i$ from that distribution. We then apply our criterion of picking the $\mathbf{u}_b$ that has the highest average fractional agreement with all other $\mathbf{u}_i$ and measure (1) the % of times we are able to retrieve the generation that has the best agreement with $\mathbf{v}$ (2) the % agreement $\mathbf{u}_b$ has with the best possible generation out of the set. We vary d, l between 2 and 50, and n between 25 and 250. All our results are based on 1000 samples. The results are in Figures 4 and 5.

For the first metric, we are able to retrieve the best generation a very high fraction of the time when l is < 5 even when d goes to higher values. Even when l is larger, we are still able to retrieve the best generation a non-trivial fraction of times – and notably our performance does not degrade much as n goes from 25 to 250.

Turning our attention to the second metric, we are able to consistently get a generation close to the best generation. This is especially true for small l where even when d increases to large values, we are able to get close to 100% agreement with the best generation. Even at high values of l however, we get relatively good agreement with the best generation – especially compared to picking a random generation – a heuristic we consistently beat.

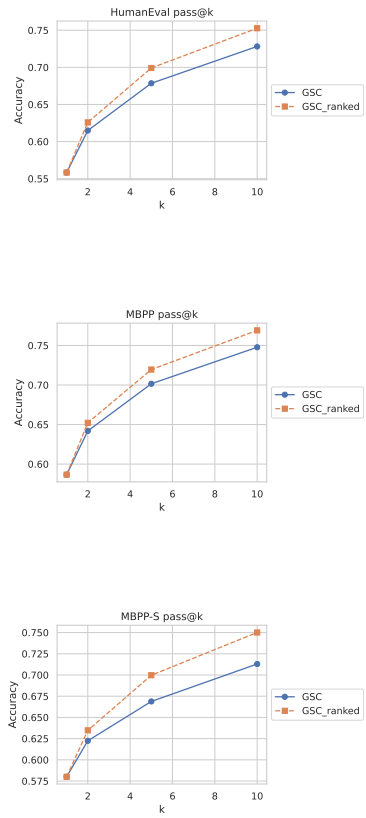


Figure 3: *pass@k* for $k > 1$ for HumanEval, MBPP, MBPP-S

C Experimental baselines

As mentioned earlier, we could not obtain Codex-001 and Codex-Cushman results on Xsum and MiniF2F due to the unexpected API shutdown. For the BLEU and Rouge-2 metrics, we report the values divided by 100. In terms of our baselines, we have

1. **Random selection** - we randomly select a generation from the set of generations
2. **Ranking by mean log probability** - we take the average log probability across the tokens in

	Medoid	Mean-logp	UCS	WUCS	Consensus-WUCS
	HumanEval				
Codex002	0.515	0.604	0.615	**0.630**	**0.633**
Codex001	0.432	0.484	0.488	**0.507**	**0.517**
Code-Cushman	0.4	0.428	0.434	**0.451**	**0.454**
Llama-13B	0.231	0.221	0.242	**0.248**	**0.25**
Llama-30B	0.29	0.286	0.324	**0.327**	**0.327**
	MBPP-S				
Codex002	0.64	0.626	**0.67**	0.643	**0.647**
Codex001	0.594	0.575	0.594	**0.599**	**0.605**
Code-Cushman	0.527	0.521	0.531	**0.541**	**0.549**
Llama-13B	**0.355**	0.331	0.340	0.344	**0.347**
Llama-30B	0.425	0.408	0.337	**0.436**	**0.438**
	MBPP				
Codex002	0.631	0.549	0.651	**0.655**	**0.659**
Codex001	0.574	0.58	0.587	**0.596**	**0.598**
Code-Cushman	0.435	0.29	0.479	**0.494**	**0.503**
Llama-13B	0.269	0.3	0.261	**0.305**	**0.304**
Llama-30B	0.346	0.332	0.351	**0.358**	**0.359**

Table 2: Mean reciprocal rank of generations for HumanEval, MBPP, MBBP-S. Best results are colored in **first**, **second**.

the generation and select the generation with the highest mean log probability

3. **Ranking using Medoid** - we take the generation with the lowest mean distance to all other generations in our confidence weighted unigram space as used in WUCS.

4. **Coder Reviewer Ranker** - This method has two variants – Normalized Reviewer (NR), and Normalized Coder Reviewer (NCR). NR computes the mean per token $\log p(x|y)$, where y is the generation and x is the prompt, and then ranks based on this metric. On the other hand, NCR merges the mean log probability ranking with NR, ranking according to $\log p(x|y) + \log p(y|x)$. As the state of the art in code reranking, these methods represent a strong baseline.

D Comparison with Coder-Reviewer Ranker

The comparison with the Code Reviewer Ranker baseline, specifically with the Normalized Reviewer (NR) and Normalized Coder-Reviewer (NCR) variants, is in Table 3. As the state of the art in code reranking, these methods represent a strong baseline. Our results demonstrate that the WUCS and Consensus-WUCS methods are highly competitive. Consensus-WUCS consistently outperforms NR and often surpasses NCR as well, despite the fact that NR and NCR require a second forward pass, which doubles the inference cost and adds latency overhead.

In the HumanEval dataset, Consensus-WUCS yields the highest accuracy for the Llama-13B and Llama-30B models. Similarly, in the MBPP-S dataset, Consensus-WUCS delivers superior performance for the Llama-13B and Llama-30B models, and closely matches the NCR for Codex models. In the MBPP dataset, the Consensus-WUCS method ranks as the best for Code-Cushman, Llama-13B, and Llama-30B models.

Notably in 40% of the experiments (6 out of 15), Consensus-WUCS outperforms all other methods, including the highly competitive NCR. Furthermore, Consensus-WUCS ranks second in 8 out of the 15 experiments, reinforcing its strong performance across diverse models and datasets.

Our results present evidence of the effectiveness of WUCS and Consensus-WUCS, which hold their own against much more heavyweight state-of-the-art methods and frequently deliver superior performance.

E Improvements are consistent across different generation temperatures

In Figure 6 (Supplement) we show how UCS reranking behaves for MBPP as the decoding sampling temperature increases. While accuracy can vary across temperatures, the ranking of the different methods remains consistent. Consensus-WUCS dominates in terms of accuracy for most of the temperature regimes until you hit the temperature of 1. Importantly, for lower temperatures where we get the best results, Both Consensus-WUCS as well as WUCS get the best accuracy. While just UCS is on par with mean log-probability ranking until a temperature of 0.4 after which it falls behind, we note that UCS does not use any probability information about the generation and thus a fair comparison would be to that of random ranking which it is consistency better than for almost the entire temperature range.

	WUCS	Consensus-WUCS	N. Reviewer	N. Coder-Reviewer
	HumanEval			
Codex002	0.558	**0.568**	0.524	**0.576**
Codex001	0.426	**0.445**	0.42	**0.482**
Code-Cushman	0.373	**0.381**	0.358	**0.385**
Llama-13B	**0.187**	**0.192**	0.164	0.181
Llama-30B	**0.263**	**0.267**	0.219	0.241
	MBPP-S			
Codex002	0.58	**0.589**	0.559	**0.595**
Codex001	0.535	**0.546**	0.509	**0.55**
Code-Cushman	0.472	**0.488**	0.455	**0.512**
Llama-13B	**0.266**	**0.277**	0.228	**0.266**
Llama-30B	**0.363**	**0.373**	0.302	0.325
	MBPP			
Codex002	0.587	**0.594**	**0.631**	0.592
Codex001	0.52	0.525	**0.532**	**0.545**
Code-Cushman	**0.405**	**0.42**	0.398	0.339
Llama-13B	0.195	**0.199**	0.185	**0.2**
Llama-30B	0.287	**0.294**	**0.289**	0.283

Table 3: Comparison with Coder-Reviewer Reranker. Best results are colored in **first**, **second**.

F Varying the maximum n-gram length does not change results

As mentioned in Section 3, UCS only considers unigrams. Here we consider Ngram Consistency Score – the more generalized version. To account for the fact that a sentence will have fewer n-grams, the more n increases, we multiply $p(t_j^{i,k})$ by $\frac{|g_i|}{|g_i|-|t_j^{i,k}|-1}$ where $t_j^{i,k}$ is now the kth appearance of the jth n-gram in the ith generation. In Figure 7 (Supplement), we show how the ranking behaves as the n increases. As can be seen, while there is a slight improvement going from $n = 1$ to $n = 4$, the improvement flattens after that point. 4-grams is also what is conventionally used when computing BLEU score so it is interesting that the same value ends up being optimal in the drastically different setting of code generation with each word being a token instead of an English word.

G Increasing number of samples maintains reranking strength

In Figure 8 (Supplement), we show how the performance changes for MBPP and Xsum as the number of samples increases. All variants of UCS are able to maintain accuracy (although Consensus-WUCS sees a drop in the beginning for Xsum but maintains its performance subsequently) even as the number of samples increases from 5 to 100. Meanwhile, the mean log probability ranking drastically declines in terms of accuracy, quickly falling below even random selection. This is likely due to the tendency of mean log probability ranking to choose degenerate sequences (Holtzman et al., 2019) which UCS variants seem to be able to avoid.

H Ada model embeddings also give a boost

To understand how generalizable the intuition behind the GCS metric (as opposed to the UCS metric) is for other similarity functions, we took the generations and used the text-ada-embedding-002 model by OpenAI to generate embedding vectors for the generations. We then used cosine similarity between the generations as the similarity function and used $GCS_{\text{Cosine Similarity}}$ to rank. The results are in Table 4. Using OpenAI embeddings as well results in improved performance over Random selection as well as mean log probability ranking validating our intuition that choosing the generation that is on average, the most similar to all other generations is a good ranking metric. That said, this particular similarity function underperforms UCS, especially for code generation so we did not investigate it further.

I Normalizing inner product degrades performance

Neural generation models are well known to generate repetitive sequences (Zhang et al., 2022; Welleck et al., 2019). In (Welleck et al., 2019), they modify the standard log-likelihood object for language models to minimize the probability of tokens immediately preceding the current token. This effectively pushes the model to generate unique new tokens and they show significant improvements in their model after they do this. If we normalize the inner product, then we would be effectively "canceling out" the contribution to the similarity score by having more unique tokens.

We evaluated the effect of normalizing the inner product by the vector norms. To understand better whether our performance is just an effect of selecting longer and more diverse sequences or whether the similarity metric itself is useful as well, we ran ablations where we evaluated ranking based on the longest sequence, as well as based on mean across the elements of $\mathbf{v}_i$ as defined in Section 3 – which takes into account the sequence diversity. The results are in Table 5 in the Supplement. Normalization results in a decline in performance. Furthermore neither ranking by the longest sequence

	Random	Mean-logp	GSC_{Ada}	Consensus-WUCS
HumanEval	0.437	**0.533**	0.487	**0.568**
MBPP	0.533	0.416	**0.579**	**0.594**
MBBP-S	0.549	0.568	**0.601**	**0.589**
MiniF2F (BLEU)	0.558	0.556	**0.584**	**0.562**
Xsum (Rouge-2)	0.197	0.214	**0.219**	**0.219**

Table 4: Performance of cosine similarity of ada embedding as the similarity function. Metric is accuracy for HumanEval, MBPP, MBPP-S and BLEU for MiniF2F. Best results are colored in **first**, **second**.

	Random	WUCS	WUCS-normalized	Longest	Most Diverse
	HumanEval				
Codex002	0.435	**0.558**	0.462	0.441	**0.51**
Codex001	0.345	**0.426**	**0.382**	0.338	0.369
Llama-30B	0.207	**0.263**	**0.235**	0.208	0.215
	Random	**WUCS**	**WUCS-normalized**	**Longest**	**Most Diverse**
	MBPP				
Codex002	0.536	**0.587**	**0.576**	0.529	0.52
Codex001	0.475	**0.52**	**0.517**	0.475	0.457
Llama-30B	0.262	**0.287**	**0.278**	0.263	0.245
	Random	**WUCS**	**WUCS-normalized**	**Longest**	**Most Diverse**
	Xsum				
Codex002	0.197	**0.215**	**0.211**	0.197	0.188
Llama-30B	0.107	**0.122**	**0.12**	0.107	0.116
GPT-J	0.065	**0.07**	**0.07**	0.065	0.069

Table 5: Impact of normalization. Best results are colored in **first**, **second**.

nor ranking by sequence diversity is sufficient to give the results we see as neither result in a consistent improvement even against the Random selection baseline.

	HumanEval	MBPP	Xsum	MiniF2F	WMT-14 French To English	WMT14 German to English
Ratio	1.95	1.34	1.21	1.08	1.07	1.08

Table 6: Diversity ratio between best and worst generations from Codex002 model for various datasets

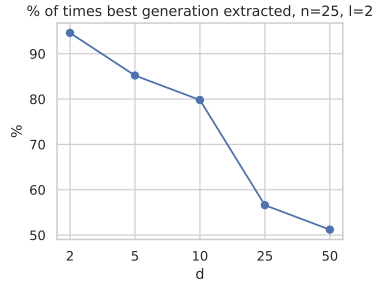
logprobs used	Medoid	Mean-logp	WUCS	Consensus-WUCS
	HumanEval			
Codex002	0.437	0.539	**0.558**	***0.568***
Codex001	0.354	0.408	**0.426**	***0.445***
Code-Cushman	0.335	0.355	**0.373**	***0.381***
Llama-13B	0.17	0.17	**0.187**	***0.192***
Llama-30B	0.225	0.228	**0.263**	***0.267***
	MBPP-S			
Codex002	**0.583**	0.57	0.580	***0.589***
Codex001	0.532	0.515	**0.535**	***0.546***
Code-Cushman	0.467	0.456	**0.472**	***0.488***
Llama-13B	**0.284**	0.27	0.266	***0.277***
Llama-30B	0.357	0.348	**0.363**	***0.373***
	MBPP			
Codex002	0.563	0.512	**0.587**	***0.594***
Codex001	0.505	0.503	**0.520**	***0.525***
Code-Cushman	0.343	0.319	**0.405**	***0.420***
Llama-13B	***0.202***	0.197	0.195	**0.199**
Llama-30B	0.276	0.273	**0.287**	***0.294***

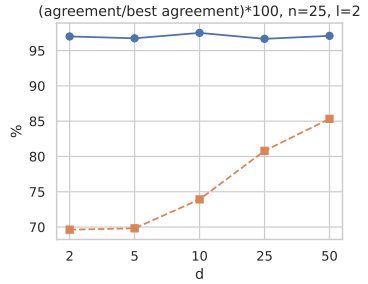
Table 7: Accuracy of generated code for HumanEval, MBPP, MBBP-S. Best results are colored in **first**, **second**. Italics for best in category (logprobs used vs not)

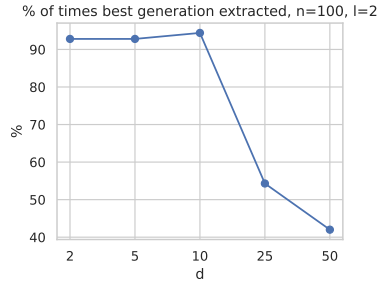
logprobs used				
	Medoid	**Mean-logp**	**WUCS**	**Consensus-WUCS**
	MiniF2F BLEU			
Codex002	**58.2**	52.9	55.8	**56.2**
Llama-13B	**24.9**	24.2	24.7	**24.8**
Llama-30B	**26.4**	25.6	25.7	25.7
GPT-J	**24.8**	24	**24.8**	**24.8**
	Xsum Rouge2			
Codex002	**21.8**	21.4	21.5	**21.9**
Llama-13B	10.3	10.3	**10.6**	**10.6**
Llama-30B	12	12.2	**12.2**	**12.3**
GPT-J	**6.9**	6.6	**7**	**6.9**
	Xsum RougeL			
Codex002	**36.3**	35.1	35.3	**35.6**
Llama-13B	20.7	20.3	**21**	**20.9**
Llama-30B	22.7	22.8	**23.1**	**23.1**
GPT-J	**17.5**	16.6	**17.8**	**17.5**
	WMT14 French → English BLEU			
Codex002	35.9	**36.6**	36.5	**37**
Llama-13B	4.2	**4.5**	**4.5**	**4.6**
Llama-30B	4	4	**4.1**	**4.1**
GPT-J	3.8	3.9	**4**	**4**
	WMT14 German → English BLEU			
Codex002	31.2	**33.2**	32.1	**34**
Llama-13B	3.1	**4**	3.5	**3.6**
Llama-30B	3.5	**3.9**	3.8	**3.9**
GPT-J	3.2	3.2	**3.3**	**3.3**

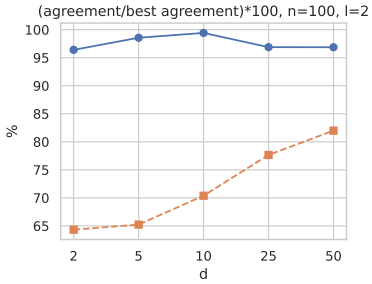
	Aqua	**Multiarith**	**StrategyQA**
Codex001	+2.8%	+4.4%	+2.5%
Codex002	-	+7.1%	+3.3%
LaMDA-137	+1.9%	+4.4%	+3.9%
UL2-20B	-1%	-0.1%	-0.1%

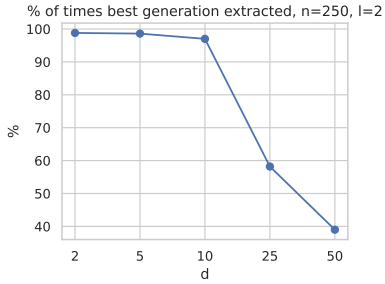
Table 8: Ratio of average GSC score for correct generations by average GSC score for incorrect generations.

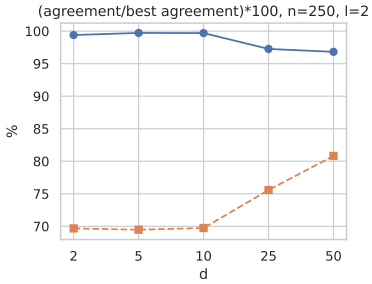

% of times best generation extracted, n=25, l=2
%
d

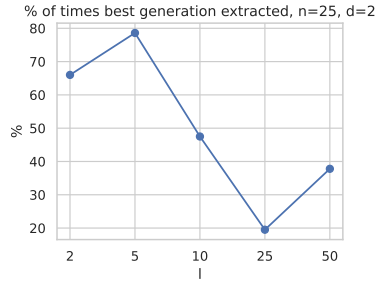

(agreement/best agreement)*100, n=25, l=2
%
d

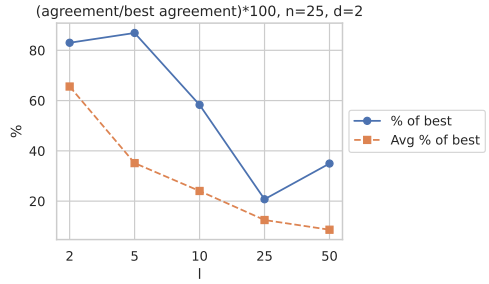

% of times best generation extracted, n=100, l=2
%
d

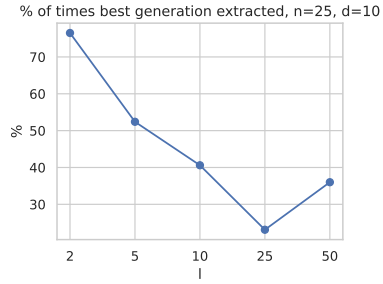

(agreement/best agreement)*100, n=100, l=2
%
d

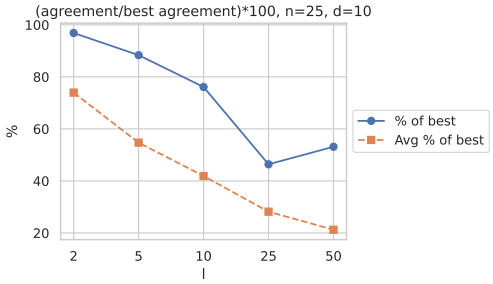

% of times best generation extracted, n=250, l=2
%
d


(agreement/best agreement)*100, n=250, l=2
%
d


% of times best generation extracted, n=25, d=2
%
l


(agreement/best agreement)*100, n=25, d=2
%
l
% of best
Avg % of best


% of times best generation extracted, n=25, d=10
%
l


(agreement/best agreement)*100, n=25, d=10
%
l
% of best
Avg % of best

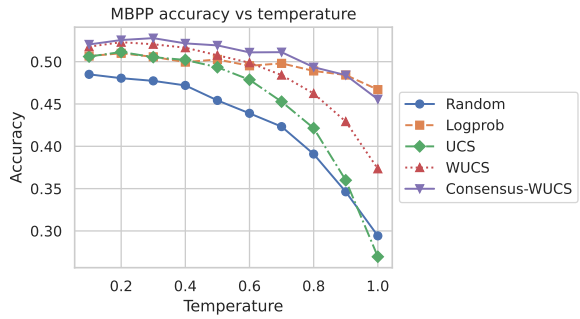


Figure 6: Accuracy for MBPP as the decoding sampling temperature increases.

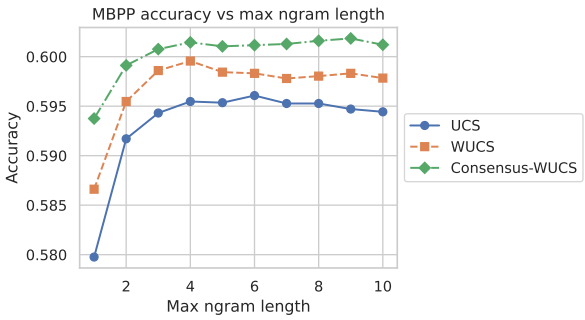


Figure 7: Accuracy for MBPP as the n in n-gram increases.

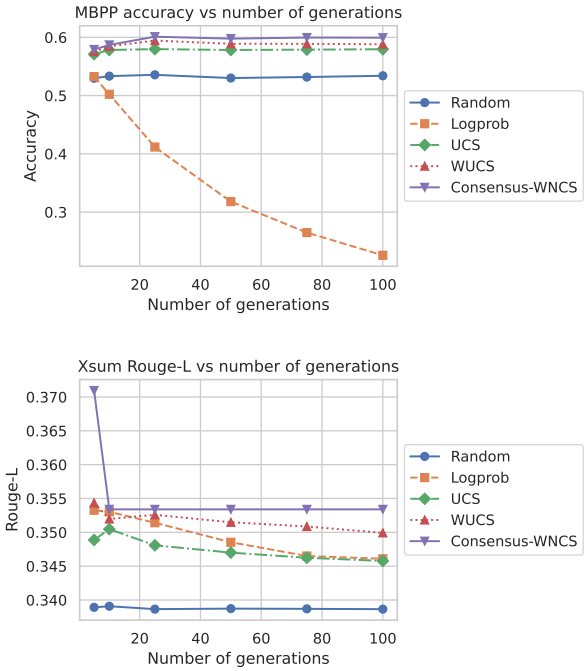


Figure 8: Accuracy for MBPP and Rouge-2 for Xsum as the number of generations increase.